

AULA 11

Construtores de Classe

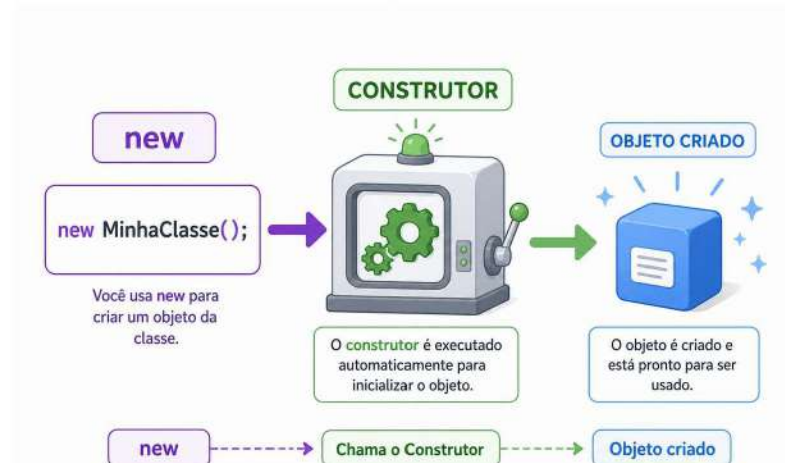
1. CONSTRUTORES DE CLASSE

Em Java, objetos são criados a partir de classes utilizando a palavra-chave `new`. A criação de um objeto segue, em geral, este formato:

```
Usuario horacio = new Usuario();
```

Parece que estamos chamando um método chamado Usuario(), por causa dos parênteses.

Um construtor contém o código que é executado quando você instancia um objeto, ou seja, o código que é executado quando você usa `new` em um tipo de classe.



Podemos escrever um construtor para nossas classes, mas se não o fizermos, o compilador escreverá um para nós. Veja como é o construtor padrão do compilador:

```
public Usuario() {  
    // código do construtor  
}
```

Se fosse um método, precisaríamos de um tipo de retorno entre "public" e "Usuario".

Seu nome é o mesmo nome da classe

A principal característica de um construtor é que ele é executado antes que o objeto possa ser atribuído a uma variável, sendo muito comum a definição de valores iniciais para seus atributos, permitindo

que os objetos já nasçam configurados, evitando a criação de instâncias “vazias” que precisam ser preenchidas atributo por atributo.

Por exemplo, na classe `Usuario`, o construtor pode ser usado para garantir que um usuário já seja criado com informações essenciais, como o nome. Com isso, garantimos que o objeto já seja criado em um estado válido e evitamos esquecer de inicializar atributos importantes.

```
public class Usuario {
    public String foto;
    public String nome;

    public Usuario(String nome) {
        this.nome = nome;
    }
}

public class TestaUsuario {
    static void main() {
        Usuario horacio = new Usuario("Horácio Augusto");
    }
}
```

Parâmetro nome para o construtor de Usuario

Passando o valor para o construtor

1.1. INTELIJ: GERAÇÃO DE CONSTRUTORES

No IntelliJ IDEA, você pode gerar um construtor automaticamente clicando com o botão direito dentro da classe Java, escolhendo a opção “Generate...” e, em seguida, selecionando “Constructor”. Depois disso, o IntelliJ exibirá uma janela com os atributos disponíveis para inclusão no construtor; basta marcar os campos desejados e confirmar para que o código seja criado automaticamente, economizando tempo e evitando erros manuais na escrita do construtor.

1.2. EXERCÍCIOS: CONSTRUTOR

- 1) Abra a classe `Usuario` e defina o construtor abaixo que receberá, obrigatoriamente, o nome do usuário. Clique com o botão direito dentro da classe, escolhendo a opção “Generate...” e, em seguida, selecione “Constructor”. Depois disso, marque o campo `nome` e confirme.

```
public class Usuario {
    public String foto;
    public String nome;

    public Usuario(String nome) {
        this.nome = nome;
    }
}
```



- 2) Agora, abra a classe `TestaAvaliacao` e observe que, na instanciação do usuário, é obrigatório informar o nome do usuário no construtor.

```
Usuario horacio = new Usuario("Horácio Augusto da Silveira");  
horacio.nome = "Horácio Augusto da Silveira";
```

- 3) Execute a classe para ver a saída.

- 4) Agora, com o uso do construtor em `Usuario`, podemos simplificar a atribuição de um usuário na avaliação. Faça:

```
static void main() {  
    Usuario horacio = new Usuario("Horácio Augusto da Silveira");  
  
    Avaliacao avaliacaoHoracio = new Avaliacao();  
    avaliacaoHoracio.usuario = new Usuario("Horácio Augusto da Silveira");  
}
```

- 7) Execute os comandos para envio das alterações ao *github*:

```
git config --global user.name "nome1-nome2"  
git config --global user.email "nome1-nome2@example.com"  
git status  
git add .  
git commit -m "Criação de construtor em Usuario"  
git push
```

AULA 12

Encapsulamento

1. APROVANDO AVALIAÇÕES

Quando criamos a classe `Avaliacao`, não determinamos como ela será aprovada. Uma avaliação será aprovada se em seu título e comentário não tiver nenhuma palavra ofensiva, ou seja, expressões, palavrões, xingamentos ou termos que desrespeitam, degradam ou ferem a dignidade de alguém. Uma forma simples de evoluir nossa classe seria criar o método `aprovar`:

```
public void aprovar() {  
    //Verificar se possui termos ofensivos  
    this.aprovado = true;  
}
```

Repare que agora estamos alterando o atributo `aprovado` com o boolean `true` para indicar se a avaliação foi aprovada ou não. No momento de usá-lo podemos fazer algo como:

```
avaliacaoHoracio.aprovar();
```

Pronto! Nossa lógica já está bem definida, agora uma avaliação para ter seus dados exibidos deve ter seu título ou comentário aprovado.

```
public void exibir() {  
    if (aprovado) {  
        System.out.println(usuario.nome);  
        System.out.println(estrelas + " estrelas " + titulo);  
        System.out.println("Avaliado em " + obterDataPorExtenso());  
        System.out.println(comentario);  
    }  
}
```

1.1. EXERCÍCIOS: APROVAÇÃO DE AVALIAÇÃO

- 1) Acesse o seu *github* e o repositório **pw2-play-livros**.
- 2) Clique no botão **Code** e copie o endereço do repositório.
- 3) Acesse o *Windows Explorer*, a unidade **D:** e a sua pasta de trabalho.
- 4) Abra um "cmd" e clone o projeto com:

```
git clone https://github.com/SEU-USUARIO/pw2-play-livros.git
```

- 5) Abra o projeto no *IntelliJ*.
- 6) Localize a classe `Avaliacao` e defina o método `aprovar()` abaixo:


```
public void aprovar() {  
    //Verificar se possui termos ofensivos  
    this.aprovado = true;  
}
```

- 7) Altere o método `exibir()` para imprimir os dados da avaliação apenas se estiver aprovada.

```
public void exibir() {  
    if (aprovado) {  
        System.out.println(" ");  
        System.out.println(usuario.nome);  
        System.out.println(estrelas + " estrelas " + titulo);  
        System.out.println("Avaliado em " + obterDataPorExtenso());  
        System.out.println(comentario);  
    }  
}
```

- 8) Execute a classe `TestaAvaliacao` para ver a saída.
- 9) Agora, altere a classe `TestaAvaliacao` para aprovar a avaliação e teste novamente.
- 10) Execute os comandos para envio das alterações ao *github*:

```
git config --global user.name "nome1-nome2"  
git config --global user.email "nome1-nome2@example.com"  
git status  
git add .  
git commit -m "Criação do método aprovar em Avaliacao"  
git push
```

2. ENCAPSULAMENTO

A nossa classe `Avaliacao` possui agora o método `aprovar()` com o comportamento que pensamos. Mas há um problema grave: nada obriga um desenvolvedor a utilizar o método `aprovar()`, ele poderia perfeitamente escrever o código desta forma em `TestaAvaliacao`:

```
avaliacaoHoracio.aprovado = true;
```

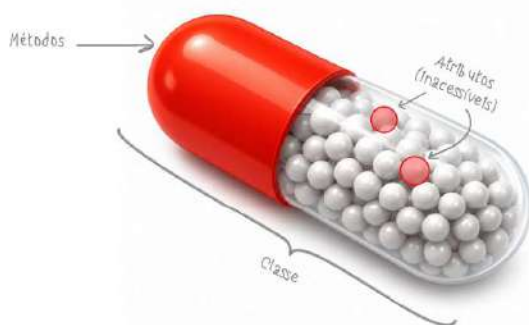
Ele consegue aprovar a avaliação, independente de nossa regra de negócio de palavras ofensivas. O problema está no fato de que o atributo `aprovado` da classe `Avaliacao` pode ser acessado e modificado diretamente por outras partes do sistema, sem nenhuma verificação ou controle.

Isso é prejudicial, pois expõe o estado interno do objeto, permitindo alterações sem respeitar as regras de negócio definidas em seus métodos. Seria como deixar cofres abertos no meio da rua — cedo ou tarde, alguém cometeria um erro ou causaria um problema.

A melhor forma de resolver isso seria forçar quem usa a classe `Avaliacao` a invocar o método `aprovar()` e não permitir o acesso direto ao atributo. Para fazer isso, basta usar **encapsulamento**: o ato de **proteger os atributos de um objeto**, restringindo o acesso direto e obrigando o uso de métodos para modificações.

2.1. O QUE É ENCAPSULAMENTO?

Encapsular significa proteger ou embrulhar os dados internos de um objeto dentro de uma "cápsula", impedindo que eles sejam acessados ou modificados diretamente. O nome reflete a ideia de isolamento: os detalhes de implementação ficam ocultos, evitando que outros objetos modifiquem o estado interno de forma imprevisível.



Assim como uma cápsula de remédio protege o princípio ativo e controla sua liberação, o encapsulamento protege os dados de um objeto e controla o acesso a eles, garantindo que toda regra de negócio seja respeitada e que os dados se mantenham consistentes.

2.2. COMO FUNCIONA O ENCAPSULAMENTO?

Pense em uma cafeteira automática. Não precisamos (e nem podemos) mexer na temperatura da água ou na pressão interna, só apertamos um botão: "Café". Toda a lógica está encapsulada dentro da máquina e ela decide quando e como fazer, garantindo que o café sempre saia da forma certa.



Para fazer isso no Java, basta declarar que os atributos não podem ser acessados de fora da classe através da palavra chave `private`:

```
public class Avaliacao {  
    public int estrelas;  
    public String titulo;  
    public LocalDateTime data;  
    public String comentario;  
    public Usuario usuario;  
    private boolean aprovado; 1 related problem  
  
    //restante do código oculto  
}
```

Marcando um atributo como privado, ninguém mais além da própria classe `Avaliacao` conseguirá acessar e modificar esse valor, fazendo com que o seguinte código não compile:

```
public class TestaAvaliacao {  
  
    static void main() {  
        Usuario horacio = new Usuario("Horácio Augusto da Silveira");  
        horacio.nome = "Horácio Augusto da Silveira";  
  
        Avaliacao avaliacaoHoracio = new Avaliacao();  
        avaliacaoHoracio.usuario = horacio;  
        avaliacaoHoracio.estrelas = 5;  
        avaliacaoHoracio.adicionar(5);  
        avaliacaoHoracio.data = LocalDateTime.of(2026, 4, 21, 13, 12, 0);  
        avaliacaoHoracio.titulo = "Livro excelente";  
        avaliacaoHoracio.comentario = "Livro excelente, muito bem escrito.";  
        avaliacaoHoracio.aprovado = true;  
  
        avaliacaoHoracio.exib  
    }  
}
```



O erro será `'aprovado' has private access in 'br.com.etechoracio.playlivros.model.Avaliacao'`. Perfeito, isso fará com que a única forma de se aprovar uma avaliação de livro seja passando pela nossa regra de negócio, que está isolada no método `aprovar`. Se alguém tentar aprovar uma `Avaliacao` e não usar `aprovar()`, vai receber um erro de compilação, já que o atributo `aprovado` é privado.

Cada classe é responsável por controlar seus atributos, portanto ela deve julgar se aquele novo valor é válido ou não! Esta validação não deve ser controlada por quem está usando a classe e sim por ela mesma, centralizando essa responsabilidade e facilitando futuras mudanças no sistema.

Repare que, quem invoca o método `aprovar` não faz a menor ideia de que existe algo que está sendo checado, basta saber o que o método faz e não como exatamente ele o faz.

O controle sobre o `aprovado` está centralizado: ninguém consegue acessá-lo sem passar por `aprovar()`, a classe `Avaliacao` é a única responsável pelos seus próprios atributos.

Na orientação a objetos, é prática quase que obrigatória proteger seus atributos com `private`.

Dessa forma, caso seja necessário modificar a maneira como as avaliações são aprovadas, a mudança será feita em apenas um ponto: o método definido na classe `Avaliacao`. Como esse comportamento está concentrado nesse local, qualquer ajuste realizado ali passa a valer automaticamente para todas as partes do sistema que utilizam esse método.

2.2.1. EXERCÍCIOS: PRIVATE

- 1) Abra a classe `Avaliacao` e defina o atributo `aprovado` com o modificador `private`:

```
public class Avaliacao {  
    public int estrelas;  
    public String titulo;  
    public LocalDateTime data;  
    public String comentario;  
    public Usuario usuario;  
    private boolean aprovado;  
  
    //restante do código oculto  
  
}
```

- 2) Abra a classe `Usuario` e defina os atributos `foto` e `nome` com o modificador `private`.

2.3. MÉTODO DE ACESSO

O modificador `private` faz com que ninguém consiga modificar e nem mesmo ler o atributo em questão. Com isso, temos um problema: como fazer para mostrar o atributo `nome` do usuário de uma `Avaliacao`, já que nem mesmo podemos acessá-lo para leitura?


```
System.out.println(usuario.nome);
```



Como esse acesso não é mais possível, precisaremos encontrar uma outra forma de realizá-lo. Sempre que precisamos arrumar uma forma de executar alguma ação com um objeto, utilizamos um método.



Para permitir o acesso aos atributos (já que eles são `private`) de uma maneira controlada, a prática mais comum é criar dois métodos, um com prefixo `get` (pegar) que retorna o valor e outro com prefixo `set` (atribuir) que muda o valor.

Uma analogia que podemos fazer é de uma caixa transparente com chave, onde o dinheiro guardado dentro da caixa representa os atributos privados de uma classe, ou seja, os dados internos que não devem ser acessados diretamente de fora do objeto.



A janela transparente simboliza os *getters*, métodos responsáveis por permitir apenas a leitura das informações. Assim como é possível enxergar o dinheiro pela parede de vidro sem abrir a caixa, os *getters* permitem visualizar os valores dos atributos sem alterar seu conteúdo.

Já a chave representa os *setters*, métodos usados para modificar os dados internos de maneira controlada. Nem todo mundo pode abrir a caixa e mexer no que está dentro dela; da mesma forma, os *setters* podem impor regras e validações antes de alterar um atributo.

A ideia principal da analogia é mostrar que os dados de um objeto devem ficar protegidos e acessíveis apenas por meio de mecanismos controlados. Isso aumenta a segurança, evita alterações indevidas e garante que o objeto mantenha suas informações consistentes ao longo da execução do programa.

Por exemplo, o nosso usuário ficaria assim, caso a gente precise dar acesso a leitura e escrita ao atributo `nome`:

```
public class Usuario {  
    private String foto;  
    private String nome;  
  
    public String getNome() {  
        return nome;  
    }  
  
    public void setNome(String nome) {  
        this.nome = nome;  
    }  
}
```

Observe que o método `getNome` não recebe nenhum parâmetro e apenas retorna o atributo `nome`, enquanto o `set` sempre recebe um parâmetro com o mesmo tipo do que o atributo que será atualizado.

É uma má prática criar uma classe e, logo em seguida, criar *getters* e *setters* para todos seus atributos. Só devemos criá-los se tiver a real necessidade em usar.

2.3.1. EXERCÍCIOS: GETTERS E SETTERS

- 1) Abra a classe `Usuario` e observe que os atributos da classe estão com `private`. A classe também possui um construtor que obriga a passagem do nome do usuário ao criar um objeto com `new`.

```
public class Usuario {  
    private String foto;  
    private String nome;
```

```
public Usuario(String nome) {  
    this.nome = nome;  
}  
}
```

- 2) Com isso, não precisamos de um método `setNome` para modificar o nome do usuário, pois já é feita na construção do objeto. Porém, precisamos de `getNome` para enxergar o valor do atributo já que é usado na classe `Avaliacao` para exibição de quem avaliou. Clique com o botão direito dentro da classe, escolhendo a opção “Generate...” e, em seguida, selecione “Getter”. Depois disso, marque o campo `nome` e confirme.

```
public class Usuario {  
    private String foto;  
    private String nome;  
  
    public Usuario(String nome) {  
        this.nome = nome;  
    }  
  
    public String getNome() {  
        return nome;  
    }  
}
```

- 3) Abra a classe `Avaliacao` e altere a instrução `usuario.nome` para `usuario.getNome()` :

```
System.out.println(usuario.nome);  
System.out.println(usuario.getNome());
```

- 4) Execute a classe `TestaAvaliacao` para ver a saída.
5) Execute os comandos para envio das alterações ao *github*:

```
git status  
git add .  
git commit -m "Encapsulamento de atributos de Usuario"  
git push
```

- 6) Abra a classe `Avaliacao` e defina todos os atributos com o modificador `private`.
7) Crie os *getters* e *setters* para os atributos que são acessados/modificados por outras classes.
8) Defina os atributos de `Livro` privados com seus respectivos *getters* e *setters* necessários.
6) Execute os comandos para envio das alterações ao *github*:

```
git status  
git add .  
git commit -m "Encapsulamento de atributos de Avaliacao e Livro"  
git push
```